

LOG430

# BrokerX - Documentation d'Architecture

Basé sur le modèle arc42

***Élève :***

**Thevenet, Louis**

Automne 2025

# Table des matières


1 Avant-propos ..... 3
2 Introduction et Objectifs ..... 3
2.1 Panorama des exigences ..... 3
2.2 Objectifs qualité ..... 3
2.3 Parties prenantes (Stakeholders) ..... 3
3 Contraintes d'architecture ..... 3
4 Portée et contexte du système ..... 4
4.1 Contexte métier ..... 4
4.2 Contexte technique ..... 6
5 Stratégie de solution ..... 6
6 Domain-Driven Design (DDD) ..... 6
6.1 Introduction au DDD ..... 6
6.2 Bounded Contexts ..... 7
6.3 Entités et Value Objects ..... 7
6.4 Aggregates ..... 8
6.5 Repositories ..... 8
6.6 Domain Services ..... 8
6.7 Ubiquitous Language ..... 9
6.8 Architecture en couches (Layered Architecture) ..... 9
7 Vue des blocs de construction ..... 10
7.1 Vue d'ensemble du système (Niveau 1) ..... 10
7.2 Architecture du domaine (Niveau 2) ..... 11
7.3 Architecture événementielle pour le traitement des ordres ..... 11
7.4 Vue d'exécution ..... 12
7.5 Vue de déploiement ..... 13
8 Concepts transversaux ..... 14
8.1 Décisions d'architecture ..... 14
8.2 Exigences qualité ..... 15
8.3 CI/CD (Intégration et Déploiement Continus) ..... 16
9 Runbook ..... 17
9.1 Vue d'ensemble des services ..... 17
9.2 Déploiement ..... 18
9.3 Surveillance et monitoring ..... 19
9.4 Tests de charge ..... 20
10 Guide d'utilisation ..... 21
10.1 Frontend Web ..... 21
10.2 API ..... 25
11 Glossaire ..... 26

# 1 Avant-propos

Les figures sont trouvables au format SVG dans le répertoire `docs/arc42/`.

# 2 Introduction et Objectifs

## 2.1 Panorama des exigences

L'application « BrokerX » est dotée d'une interface web faisant appel à une passerelle API sécurisée faisant elle-même appel à différents micro-services. Elle permet aux particuliers et petites entreprises d'accéder à une plateforme de courtage moderne. Les clients peuvent ainsi :

- Gérer un portefeuille personnel (solde, historique),
- Passer des ordres d'achat/vente d'instruments financiers (actions),
- Consulter les données de marché en temps réel.

## 2.2 Objectifs qualité

| Priorité | Objectif qualité | Scénario |
|---|---|---|
| 1 | **Disponibilité** | $99,9\%$ |
| 2 | **Performance** | Latence P95 (ordre $\rightarrow$ ACK) $\leq 100$ ms & Débit $\geq 1200$ ordres/s |
| 3 | **Observabilité** | Logs + métriques (4 Golden Signals) |

Tableau 1. – Objectifs qualité

## 2.3 Parties prenantes (Stakeholders)

- **Clients** : utilisateurs via interface web.
- **Opérations Back-Office** : gestion des règlements, supervision.
- **Risque** : surveillance pré- et post-trade.
- **Fournisseurs de données de marché** : cotations en temps réel et simulateur d'exécution.

# 3 Contraintes d'architecture

| Contrainte | Description |
|---|---|
| **Technologie** | Les plateformes/langages serveur privilégiées : Java, C#, Go, Rust ou C++ |

Tableau 2. – Contraintes d'architecture

# 4 Portée et contexte du système

## 4.1 Contexte métier

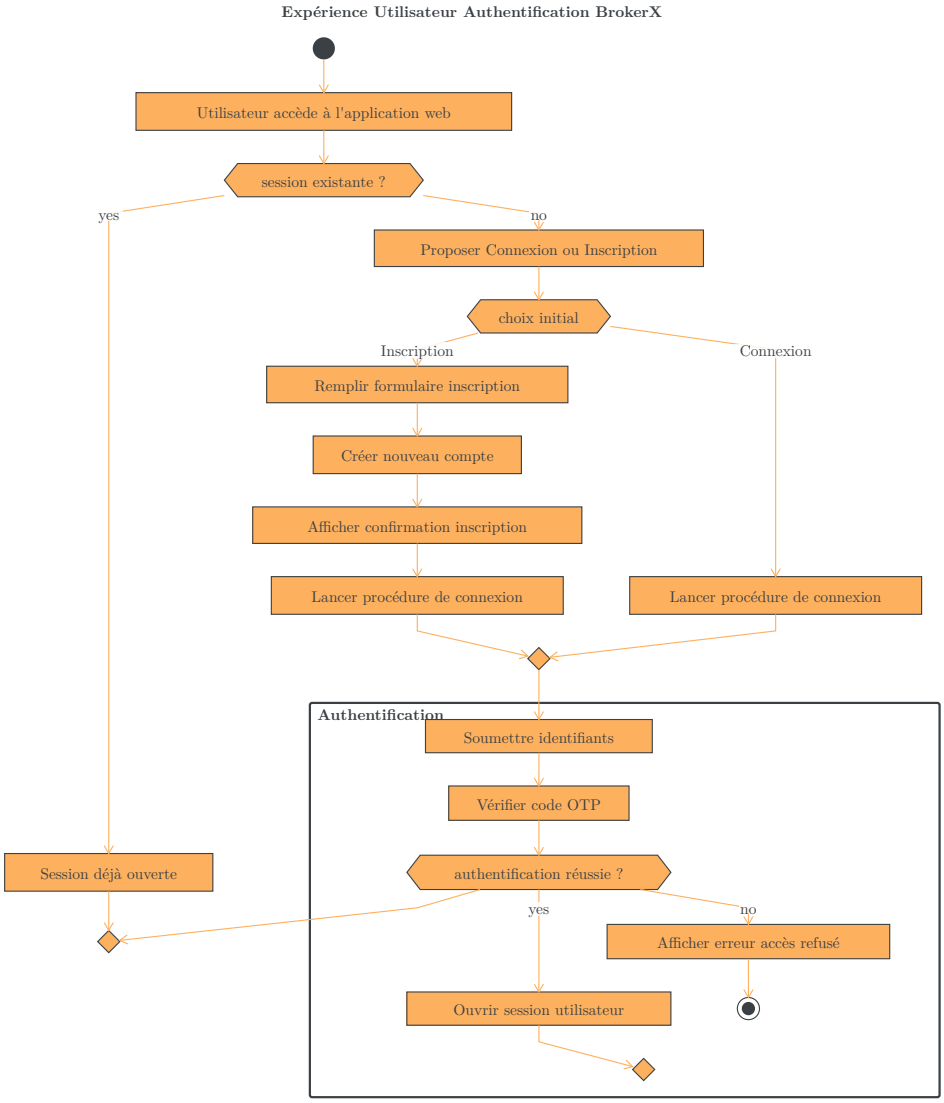



Fig. 1. – Authentication Activity Diagram

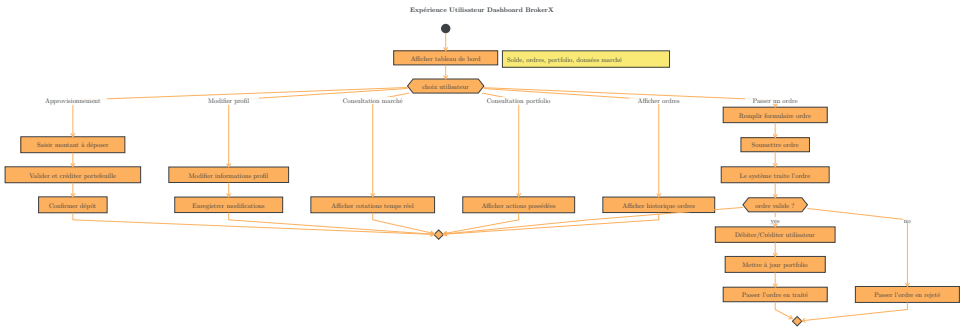



Fig. 2. – Dashboard Activity Diagram

Le système permet aux utilisateurs de :

- Gérer leur compte utilisateur
- Approvisionner leur compte virtuel

- Passer des ordres d'achat/vente
- Consulter l'état des ordres et le portefeuille
- Consulter les données du marché

#### 4.1.1 Priorisation MoSCoW

- Must Have
  - UC-01 — Inscription & Vérification d'identité
  - UC-03 — Approvisionnement du portefeuille (dépôt virtuel)
  - UC-05 — Placement d'un ordre (marché/limite) avec contrôles pré-trade
- Should Have
  - UC-07 — Appariement interne & Exécution (matching)
  - UC-06 — Modification / Annulation d'un ordre
- Could Have
  - UC-08 — Confirmation d'exécution et Notifications
  - UC-02 — Authentification & MFA
  - UC-04 — Abonnement aux données de marché
- Won't Have

#### 4.1.2 UC-01 — Inscription & Vérification d'identité

Ce cas d'utilisation est le point d'entrée d'un nouvel utilisateur sur la plateforme BrokerX. L'inscription comprend les étapes suivantes :

- Le client remplit un formulaire d'inscription avec des informations de base (email, mot de passe, etc).
- Le système crée son compte utilisateur avec le statut « Non vérifié »
- Il sera vérifié à la première connexion, et en attendant il ne pourra pas effectuer d'actions.

#### 4.1.3 UC-03 — Approvisionnement du portefeuille (dépôt virtuel)

Ce cas d'utilisation permet à un utilisateur de déposer des fonds sur son compte BrokerX. Le processus comprend les étapes suivantes :

- L'utilisateur se connecte à son compte.
- Il navigue vers la section « Approvisionnement ».
- Il saisit le montant à déposer.
- Le système valide le montant (positif, limites).
- Le système crédite le portefeuille de l'utilisateur avec le montant déposé. (dépôt virtuel)
- Le système affiche confirmation

#### 4.1.4 UC-05 — Placement d'un ordre (marché/limite) avec contrôles pré-trade

Ce cas d'utilisation permet à un utilisateur de passer un ordre d'achat ou de vente d'instruments financiers. Le processus comprend les étapes suivantes :

- L'utilisateur se connecte à son compte.
- Il navigue vers la section « Passer un ordre ».
- Il remplit le formulaire d'ordre avec les détails (type, quantité, prix, etc).
- Le système valide le formulaire (champs obligatoires, formats).
- La demande d'ordre est envoyée
- Le système effectue des contrôles pré-trade (solvabilité, limites de risque).
- L'utilisateur peut consulter l'état de son ordre en direct
- Le système traite l'ordre et met à jour son état (exécuté, rejeté, annulé).
- Le système notifie l'utilisateur du résultat de l'ordre.

#### 4.1.5 UC-07 — Appariement interne & Exécution (matching)

#### 4.1.6 UC-06 — Modification / Annulation d'un ordre

Ce cas d'utilisation permet à un utilisateur de modifier ou d'annuler un ordre en attente. Pour chaque ordre en statut « Pending », l'utilisateur peut l'annuler ou le modifier (quantité, prix limite et type). Le système valide la demande et met à jour l'état de l'ordre en conséquence. (En réalité les ordres sont traités très rapidement donc cette fonctionnalité est difficile à utiliser)

#### 4.1.7 UC-08 — Confirmation d'exécution et Notifications

Ce cas d'utilisation permet à un utilisateur de recevoir des confirmations d'exécution pour ses ordres passés. Lorsqu'un ordre est exécuté, le système envoie un email à l'utilisateur avec les détails de l'exécution (quantité, prix, frais, etc).

#### 4.1.8 UC-02 — Authentification & MFA

Ce cas d'utilisation permet à un utilisateur de se connecter de manière sécurisée à son compte BrokerX en utilisant l'authentification multi-facteurs (MFA). Le processus comprend les étapes suivantes :

- L'utilisateur accède à la page de connexion.
- Il saisit son email et son mot de passe.
- Le système valide les informations d'identification.
- Si les informations sont correctes, le système envoie un code OTP par email à l'utilisateur.
- L'utilisateur saisit le code OTP reçu.
- Le système valide le code OTP.
- Le système transmfet un token JWT pour les sessions authentifiées.

#### 4.1.9 UC-04 — Abonnement aux données de marché

### 4.2 Contexte technique

- **Applications clientes** : Postman, frontend web
- **Couche API**: Axum (Rust) pour les endpoints RESTful
- **Couche base de données** : PostgreSQL
- **API Gateway** : Axum (Rust)
- **Communication** : Requêtes HTTP/HTTPS entre clients, gateway et microservices

## 5 Stratégie de solution

| Problème | Approche de solution |
|---|---|
| **Performance** | Utilisation du langage Rust |
| **Gestion des données** | Base relationnelle (**PostgreSQL**), cache en mémoire pour les évènements et les données fréquemment utilisées |
| **Observabilité** | Métriques Promethus et Dashboards Grafana |

Tableau 3. – Stratégie de solution

## 6 Domain-Driven Design (DDD)

### 6.1 Introduction au DDD

BrokerX applique les principes du Domain-Driven Design pour structurer le code métier autour des concepts du domaine financier. Cette approche permet une meilleure séparation des préoccupations et une évolutivité accrue du système.

## 6.2 Bounded Contexts

Le système BrokerX est divisé en plusieurs contextes délimités (Bounded Contexts), chacun représentant un sous-domaine métier distinct :

| **Bounded Context** | **Responsabilité** | **Service** |
|---|---|---|
| **User Context** | Gestion des utilisateurs, authentification, soldes et positions | UserService |
| **Order Context** | Gestion du cycle de vie des ordres | OrderService |
| **Market Data Context** | Données de marché, cotations, exécution simulée | MarketDataService |
| **Event Store Context** | Event sourcing, saga chorégraphiée, outbox pattern transactionnel | EventStoreService |
| **Notification Context** | Notifications d'exécution d'ordres, alertes utilisateur | NotificationService |

Tableau 4. – Bounded Contexts de BrokerX

Chaque contexte est implémenté comme un microservice autonome avec sa propre base de données, garantissant l'isolation et l'indépendance des domaines.

## 6.3 Entités et Value Objects

### 6.3.1 Entités principales

Les entités sont des objets métier identifiés par un identifiant unique (UUID) dont l'identité persiste dans le temps :

**User Context**

- `User` : Entité représentant un utilisateur du système
  - Attributs : `id` (UUID), `email`, `password_hash`, `balance`, `verified`, `created_at`
  - Invariants : email unique,
- `Holding` : Position d'un utilisateur sur un instrument
  - Attributs : `user_id`, `symbol`, `quantity`, `average_price`
  - Invariants : quantity $> 0$

**Order Context**

- `Order` : Entité centrale représentant un ordre de bourse
  - Attributs : `id` (UUID), `user_id`, `symbol`, `order_type`, `quantity`, `price`, `status`, `created_at`
  - Machine à états : `PENDING` $\rightarrow$ `VALIDATED` $\rightarrow$ `FILLED` / `REJECTED` / `CANCELLED`
  - Invariants : quantity $> 0$, price $> 0$ (pour ordres limites)

**Market Data Context**

- `Stock` : Instrument financier
  - Attributs : `symbol`, `name`, `current_price`, `last_updated`
  - Invariants : price $> 0$

### 6.3.2 Value Objects

Les Value Objects sont des objets immuables définis par leurs attributs :

- `OrderType` : Énumération `MARKET` | `LIMIT`
- `OrderStatus` : Énumération `PENDING` | `VALIDATED` | `FILLED` | `REJECTED` | `CANCELLED`
- `Side` : Énumération `BUY` | `SELL`

- `Money` : Représentation du montant avec précision (Decimal)

**Event Objects (Value Objects spécialisés)**
- `OrderEvent` : Événement immutable dans l'event store
- `OrderEventPayload` : Payload d'événement pour la communication Redis
- `OrderEventType` : Type d'événement dans la saga (`ORDER_CREATED`, `ORDER_VALIDATED`, etc.)

Les événements sont des Value Objects car ils sont :
- Immutables une fois créés (pas de modification après insertion)
- Définis entièrement par leur contenu (event_type, order_id, données métier)
- Utilisés pour la communication et l'audit, sans identité métier propre

## 6.4 Aggregates

Les aggregates définissent les frontières transactionnelles et de cohérence :

**User Aggregate**
- Racine : `User`
- Entités liées : `Holding[]`
- Règles : Toute modification du solde ou des positions doit passer par l'aggregate User pour garantir la cohérence

**Order Aggregate**
- Racine : `Order`
- Auto-suffisant : un ordre est traité de manière autonome
- Règles : Les transitions d'état sont contrôlées par la machine à états

## 6.5 Repositories

Le pattern Repository abstrait l'accès aux données et fournit une interface orientée collection :

```
trait UserRepository {
    fn find_by_id(&self, id: Uuid) -> Option<User>;
    fn find_by_email(&self, email: &str) -> Option<User>;
    fn save(&self, user: &User) -> Result<()>;
    fn update_balance(&self, user_id: Uuid, amount: Decimal) -> Result<()>;
}

trait OrderRepository {
    fn find_by_id(&self, id: Uuid) -> Option<Order>;
    fn find_by_user(&self, user_id: Uuid) -> Vec<Order>;
    fn save(&self, order: &Order) -> Result<()>;
    fn update_status(&self, id: Uuid, status: OrderStatus) -> Result<()>;
}
```

Chaque bounded context possède son propre repository isolé, implémenté avec PostgreSQL.

## 6.6 Domain Services

Les services de domaine encapsulent la logique métier qui ne correspond pas naturellement à une entité :

**TradeValidator**
- Responsabilité : Validation pré/post-trade des ordres
- Logique :
  - Vérification de solvabilité (solde suffisant pour achat)
  - Vérification des positions (quantité suffisante pour vente)

    - Contrôle des bandes de prix
- Collaboration : Interroge `UserRepository` et `MarketDataRepository`

**ExecutionSimulator**
- Responsabilité : Simulation de l'exécution sur le marché
- Logique : Interaction avec les autres services pour simuler l'exécution des ordres

## 6.7 Ubiquitous Language

Le **ubiquitous language** utilisé dans BrokerX reflète le vocabulaire métier de la finance :

| **Terme métier** | **Signification dans le code** |
|---|---|
| Ordre | `Order` entity |
| Exécution | Passage d'un ordre de `VALIDATED` à `FILLED` |
| Position | `Holding` entity |
| Solde | `User.balance` |
| Pré/Post-trade | Validation avant/après exécution (`TradeValidator`) |

Tableau 5. – Ubiquitous Language de BrokerX

## 6.8 Architecture en couches (Layered Architecture)

Bien que BrokerX utilise une architecture microservices, chaque service respecte une séparation en couches :

1. API Layer (HTTP Handlers) → Axum routes
2. Application Layer (Use Cases) → Orchestration
3. Domain Layer (Business Logic) → Entities, Services
4. Infrastructure Layer (Repos) → PostgreSQL, HTTP clients

**Application Layer** : Cas d'usage orchestrant les opérations du domaine

**Infrastructure Layer** : Implémentations concrètes des repositories et clients externes (mail par exemple)

Cette séparation permet de tester la logique métier indépendamment de l'infrastructure.

# 7 Vue des blocs de construction

## 7.1 Vue d'ensemble du système (Niveau 1)

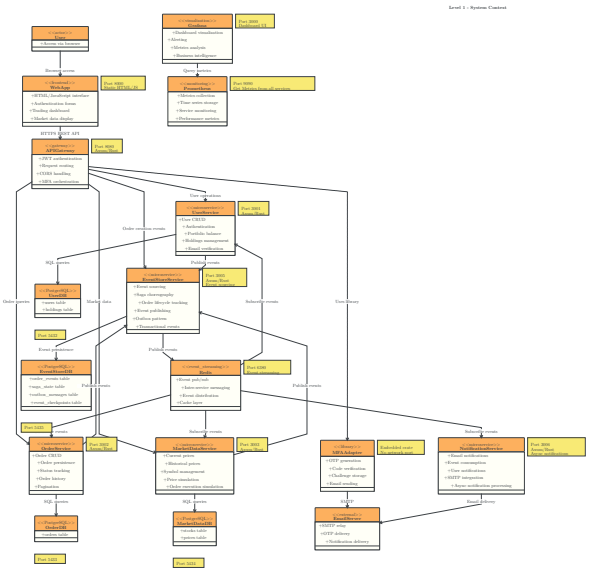


Fig. 3. – Vue d'ensemble du système

Le système BrokerX suit une architecture restAPI avec passerelle API et microservices :

1. La passerelle API est le point d'entrée unique pour les clients.
2. La passerelle API orchestre l'authentification via MFA (→ token JWT)
3. Les différents services sont :
   - Service Utilisateurs : gestion des comptes et portefeuilles utilisateurs
   - Service Ordres : gestion des ordres en mémoire persistente
   - Service Données de Marché : récupération des cotations en temps réel, execution des ordres sur le marché
   - Service Event Store : event sourcing, outbox pattern transactionnel
   - Service Notifications : envoi d'emails (Code OTP, confirmation d'exécution)
4. Prometheus collecte les métriques de tous les services
5. Grafana visualise les métriques via dashboards

## 7.2 Architecture du domaine (Niveau 2)

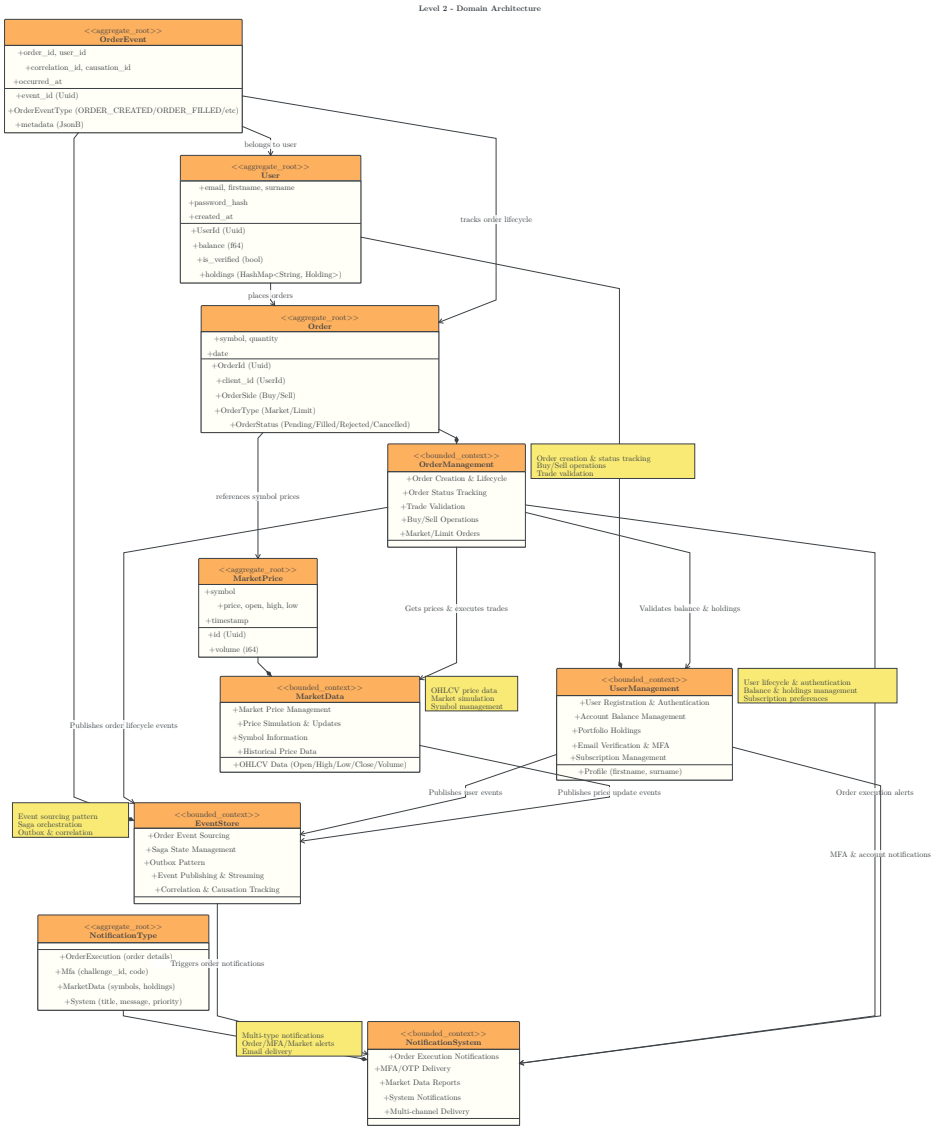



Fig. 4. – Architecture du domaine

Ce diagramme présente les 5 bounded contexts métier et leurs aggregate roots, avec les relations inter-domaines via événements pour maintenir la cohérence distribuée.

## 7.3 Architecture événementielle pour le traitement des ordres

Le système utilise une saga chorégraphiée distribuée pour traiter les ordres de manière asynchrone et résiliente. Le processus se décompose en deux phases :

1. Acceptation synchrone : L'API Gateway accepte l'ordre et retourne immédiatement un `202 Accepted`
2. Traitement asynchrone : Une saga distribuée coordonne la validation, l'exécution et le règlement via événements Redis

## 7.4 Vue d'exécution

### 7.4.1 Parcours utilisateur complet

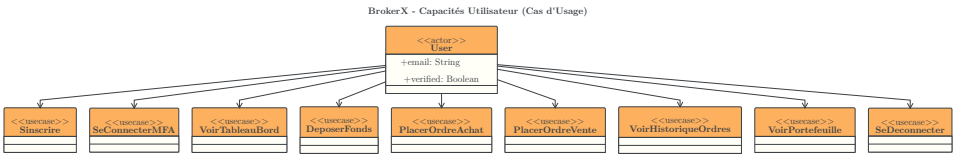



Fig. 5. – Parcours utilisateur complet

Ce diagramme de séquence illustre l'ensemble des interactions utilisateur dans le système BrokerX, de l'inscription initiale jusqu'à la gestion complète du portefeuille. Il couvre tous les cas d'usage implémentés :

### 7.4.2 Flux principaux documentés

1. **Inscription et vérification d'identité (UC-01)**
   - Création de compte avec validation email
   - Processus de vérification par code à 6 chiffres
   - Activation du compte utilisateur
2. **Authentification et MFA (UC-02)**
   - Connexion sécurisée avec email/mot de passe
   - Authentification multi-facteurs obligatoire
   - Génération et validation de tokens JWT
3. **Approvisionnement du portefeuille (UC-03)**
   - Dépôt virtuel de fonds
4. **Placement d'ordres avec contrôles pré-trade (UC-05)**
   - Validation des formulaires d'ordre
   - Contrôles de risque et de solvabilité
   - Traitement asynchrone par le service d'ordres
5. **Modification et annulation d'ordres (UC-06)**
   - Modification des ordres en statut PENDING (quantité, prix limite, type)
   - Annulation immédiate des ordres non encore exécutés
   - Validation des autorisations utilisateur
6. **Appariement interne et exécution (UC-07)**
   - Simulation de matching sur le marché via MarketDataService
   - Mise à jour temps réel des statuts d'ordre (PENDING → VALIDATED → FILLED)
7. **Confirmation d'exécution et notifications (UC-08)**
   - Envoi automatique d'emails de confirmation post-exécution
   - Notifications incluant détails complets (quantité, prix, frais, nouveau solde)
   - Traçabilité des tentatives d'envoi et statuts de livraison
8. **Consultation des données de marché (UC-04)**
   - Accès aux cotations en temps réel des instruments financiers
   - Historique des prix et données de marché

- Mise à jour périodique des prix (cycle de 5 secondes)

## 7.5 Vue de déploiement

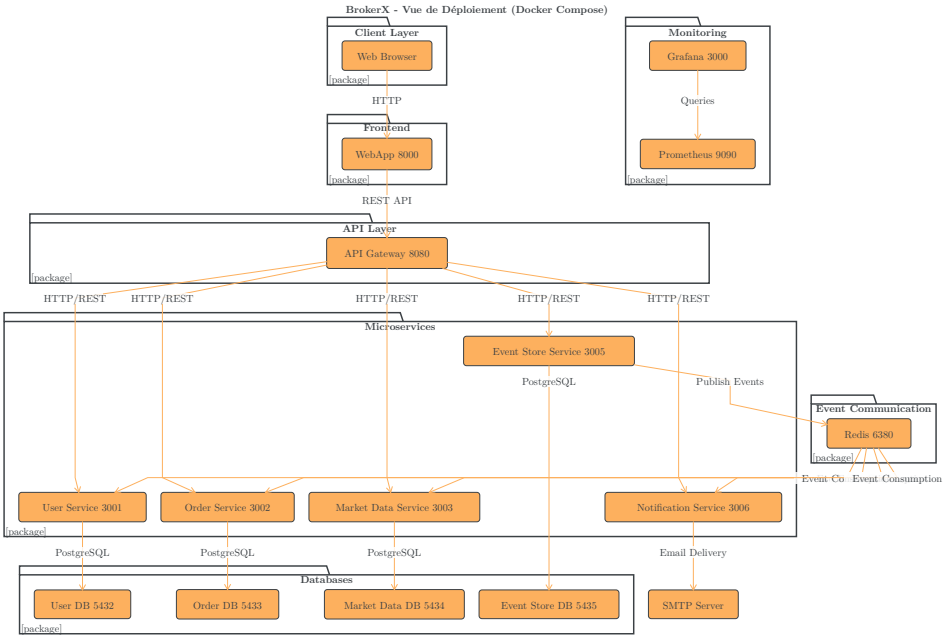



Fig. 6. – Vue de déploiement

L'architecture de déploiement utilise **Docker Compose** pour orchestrer l'ensemble des composants en conteneurs. La stack complète comprend 6 microservices, 4 bases de données PostgreSQL, Redis comme message broker, et une stack de monitoring complète.

Le diagramme met en évidence la communication événementielle via Redis ainsi que les connexions directes API Gateway vers microservices.

Les requêtes Prometheus → `/metrics` (de chaque service) ne sont pas représentées par souci de lisibilité.

### 7.5.1 Couche Frontend

- **WebApp 8000** : Interface web statique servie via serveur HTTP, accessible par les navigateurs clients

### 7.5.2 Couche API

- **API Gateway** (port 8080) : Point d'entrée unique orchestrant toutes les requêtes clients vers les microservices via HTTP/REST

### 7.5.3 Couche Microservices

- **User Service** (port 3001) : Gestion des utilisateurs et authentification, consomme les événements Redis pour mise à jour des soldes
- **Order Service** (port 3002) : Gestion des ordres avec event consumer Redis pour traitement des validations
- **Market Data Service** (port 3003) : Données de marché et simulation d'exécution, consomme les événements pour déclencher les exécutions
- **Event Store Service** (port 3005) : Event sourcing central, publie tous les événements vers Redis via transactional outbox

- **Notification Service** (port 3006) : Envoi d'alertes et notifications par email, connecté au serveur SMTP externe

#### 7.5.4 Couche Données

Chaque service possède sa propre base de données PostgreSQL 15 pour l'isolation :

- **User DB** (port 5432) : Données utilisateurs et holdings
- **Order DB** (port 5433) : Ordres avec table `processed_events` pour idempotence
- **Market Data DB** (port 5434) : Instruments financiers et historique des prix
- **Event Store DB** (port 5435) : Tables `order_events`, `saga_state`, `outbox` pour event sourcing

#### 7.5.5 Communication Événementielle

- **Redis** (port 6380) : Message broker central recevant les événements de l'EventStoreService et les distribuant aux event consumers de chaque microservice
- **Event Flow** : EventStoreService → Redis → (OrderService, MarketDataService, UserService, NotificationService)
- **Transactional Outbox Worker** : Traitement périodique de l'outbox dans EventStoreService (polling 100ms)
- **Event Consumers** : Abonnements Redis dans chaque service pour saga chorégraphiée distribuée

#### 7.5.6 Monitoring

- **Prometheus** (port 9090) : Collecte des métriques via endpoints `/metrics` de tous les microservices
- **Grafana** (port 3000) : Visualisation des métriques via requêtes Prometheus et dashboards pré-configurés
- **K6** : Des scénarios de tests de charge sont disponibles

#### 7.5.7 Services Externes

- **SMTP Server** : Serveur de messagerie externe pour l'envoi des emails MFA et notifications d'exécution

#### 7.5.8 Réseau et Communication

- **Réseau Docker** : Tous les services communiquent via un réseau privé `brokerx-network`
- **Communication synchrone** : Client → WebApp → API Gateway → Microservices (HTTP/REST)
- **Communication asynchrone** : EventStoreService → Redis → Event Consumers (saga événementielle)
- **Connexions BDD** : Chaque microservice avec sa base PostgreSQL dédiée

# 8 Concepts transversaux

## 8.1 Décisions d'architecture

Les décisions architecturales majeures sont documentées sous forme d'ADR (Architecture Decision Records) dans le répertoire `docs/adr/`.

#### 8.1.1 ADR documentées

1. **ADR 001** Choix de l'architecture web et frontend (API Gateway + SPA)
2. **ADR 002** Adoption des UUIDs comme identifiants pour clients et ordres
3. **ADR 003** Choix de la technologie de logs (tracing pour journaux structurés)
4. **ADR 004** Implémentation du traitement des ordres avec saga chorégraphiée

5. **ADR 005** Architecture microservices (5 services distincts + API Gateway)
6. **ADR 006** Authentification JWT avec MFA (multi-factor authentication)
7. **ADR 007** Transactional Outbox Pattern pour publication fiable d'événements
8. **ADR 008** Event Sourcing pour audit immutable et traçabilité complète

Chaque ADR décrit le contexte, la décision retenue et ses conséquences.

## 8.2 Exigences qualité

BrokerX est capable de gérer $\geq$ 1800 ordres/s en pic. La latence P95 sur l'endpoint `/v1/orders` est garantie grâce à la queue asynchrone dans l'orchestrateur d'ordres. Même lors de pics de charge et de perte de disponibilité sur certains services, les utilisateurs peuvent toujours passer des ordres qui seront traités en différé.

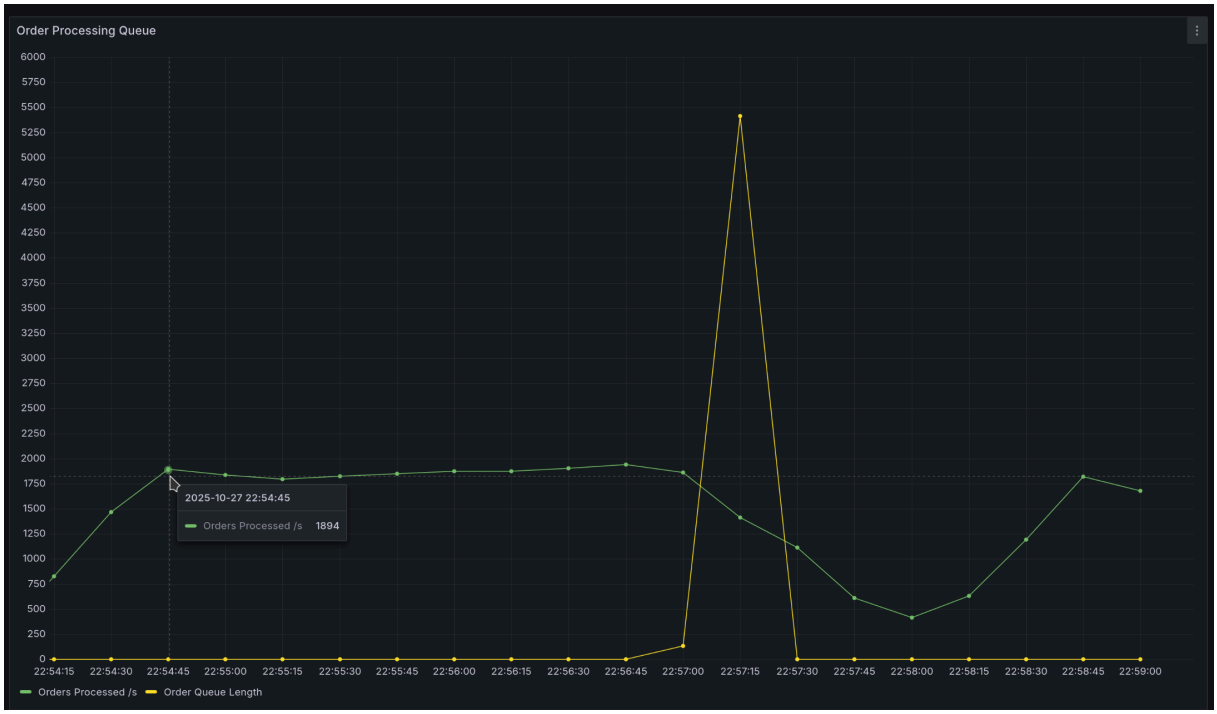



Fig. 7. – Ordres traités par seconde et taille de la queue

On voit que la queue d'ordres monte en pic lors des charges élevées, mais les ordres sont toujours acceptés rapidement par l'API Gateway.

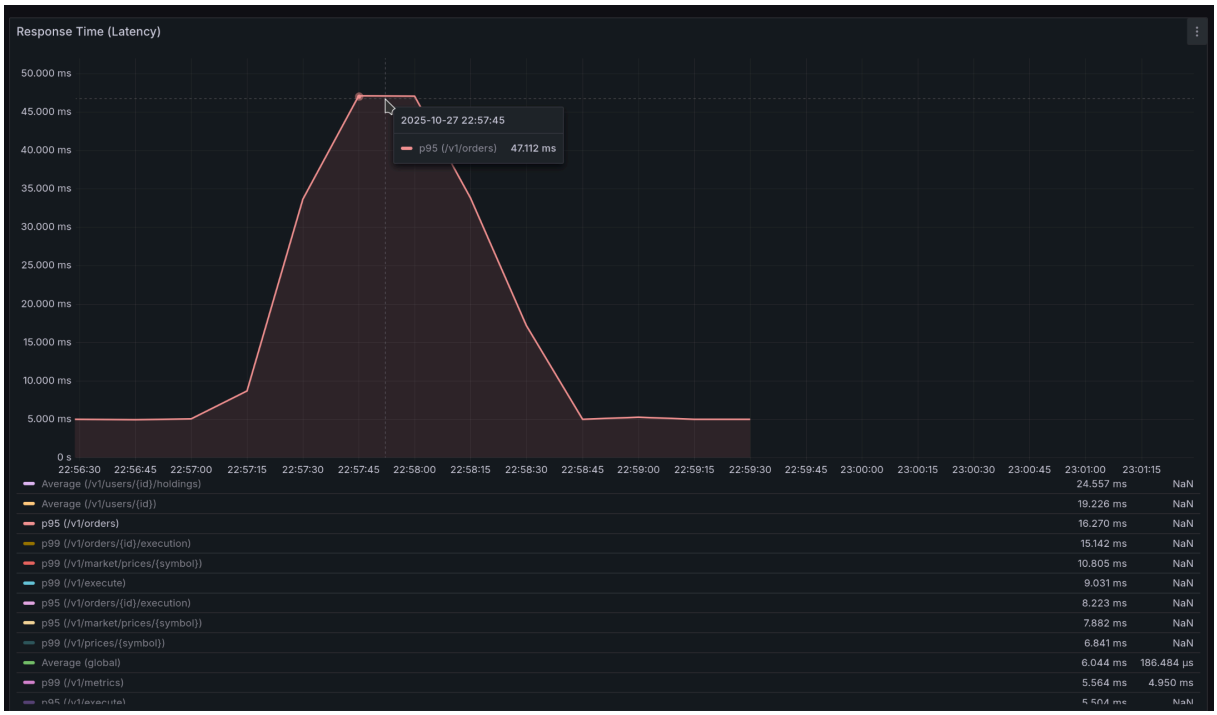



Fig. 8. – Latence P95 sur l'endpoint /v1/orders

Le premier service à tomber est celui chargé de mettre à jour les soldes et portefeuilles des utilisateurs. Mais le passage d'ordre est maintenu.

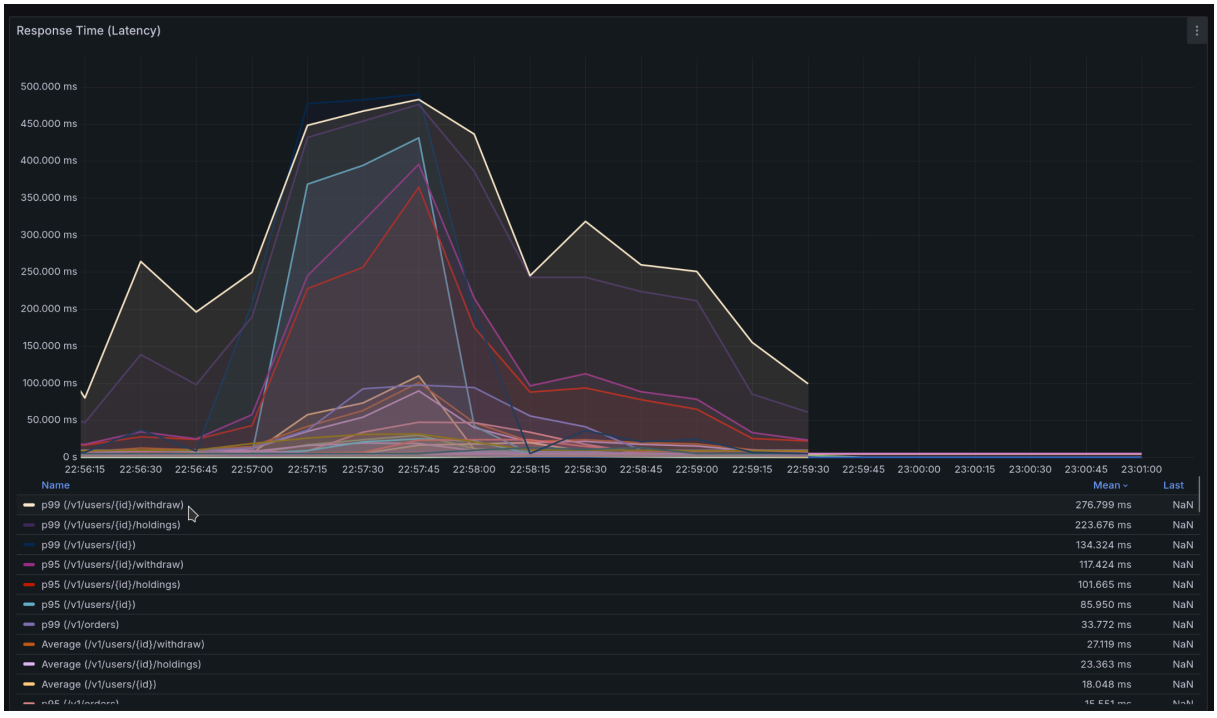



Fig. 9. – Latence sur tous les endpoints

## 8.3 CI/CD (Intégration et Déploiement Continus)

BrokerX utilise **GitHub Actions** pour l'automatisation CI/CD avec trois pipelines principaux :

| Pipeline | Déclencheur | Actions |
|---|---|---|
| CI | Push sur `main` et PRs | Tests + formatage Rust |
| Release | Tags `v*.*.*` | Build multi-plateformes via `cargo-dist` |
| Bump | Manuel | Incrémentation de versions |

Tableau 6. – Pipelines CI/CD

#### 8.3.1 Pipeline CI

Le workflow CI déploie 3 bases PostgreSQL en parallèle (user-db:5432, order-db:5433, market-data-db:5434), initialise les schémas SQL et exécute `cargo test --all-features --workspace`. Le formatage est vérifié via `cargo fmt --all --check`.

**Optimisations** : Cache Rust (`Swatinem/rust-cache@v2`), healthchecks PostgreSQL, temps d'exécution 5-8 minutes.

#### 8.3.2 Pipeline Release

Déclenché par tags Git, `cargo-dist` génère automatiquement les binaires multi-plateformes, archives (`.tar.gz`, `.zip`), checksums SHA256 et notes de release. Publication sur GitHub Releases.

#### 8.3.3 Outils de développement

Le `Justfile` fournit des raccourcis pour Docker Compose :

```
just docker-up           # Démarre toute la stack
just docker-services     # Microservices uniquement
just docker-monitoring   # Prometheus + Grafana
```

#### 8.3.4 Stratégie Git

Workflow simplifié : branche `main` protégée par CI, feature branches, validation par PR avant merge, releases via tags

# 9 Runbook

Le runbook suivant documente les procédures opérationnelles standards (SOP) pour le déploiement, la surveillance et le dépannage du système BrokerX.

## 9.1 Vue d'ensemble des services

BrokerX est composé de 6 microservices principaux, 4 bases de données PostgreSQL, Redis et une stack de monitoring (Prometheus + Grafana). Par défaut:

| Service | Port | Endpoint santé |
|---|---|---|
| Web App | 8000 | `http://localhost:8000` |
| API Gateway | 8080 | `http://localhost:8080/v1/health` |
| User Service | 3001 | `http://localhost:3001/v1/health` |
| Order Service | 3002 | `http://localhost:3002/v1/health` |
| Market Data Service | 3003 | `http://localhost:3003/v1/health` |
| Event Store Service | 3005 | `http://localhost:3005/v1/health` |
| Notification Service | 3006 | `http://localhost:3006/v1/health` |
| Redis | 6380 | Connexion TCP |
| Prometheus | 9090 | `http://localhost:9090` |
| Grafana | 3000 | `http://localhost:3000` |

Tableau 7. – Services BrokerX et ports associés

## 9.2 Déploiement

### 9.2.1 Prérequis

- Docker Engine 20.10+
- Docker Compose 2.0+
- Just (gestionnaire de commandes) — optionnel mais recommandé

### 9.2.2 Configuration initiale

1. Fichier de configuration
   - Un fichier `.env` devrait êter fourni avec l'archive
   - Sinon créer le fichier de configuration `.env`:

     ```
     cp .env.example .env
     ```
2. Choisir un mode MFA :
   - La passerelle API peut être configurée pour ne pas envoyer de mails et accepter tous les codes OTP avec: `- MFA_BYPASS=false` dans la section api-gateway du fichier `docker-compose.yml`.
   - Modifier `.env` avec les variables d'environnement appropriées (entre autres)
     - `SMTP_HOST` : serveur SMTP pour l'envoi d'emails MFA
     - `SMTP_PORT` : port SMTP (587 par défaut)
     - `SMTP_USERNAME` : identifiant SMTP
     - `SMTP_PASSWORD` : mot de passe SMTP
     - `SMTP_FROM` : adresse email d'envoi

### 9.2.3 Démarrage complet de la stack

**Méthode 1 : Avec Just (recommandé)**

```
just docker-up
```

Cette commande démarre tous les services et affiche les URLs d'accès.

**Méthode 2 : Avec Docker Compose**

```
docker compose up -d
```

### 9.2.4 Démarrage sélectif des composants

**Démarrer uniquement les bases de données :**

```
just docker-db
# ou
docker compose up -d user-db order-db market-data-db
```

**Démarrer uniquement les microservices :**

```
just docker-services
# ou
docker compose up -d user-service order-service \
  market-data-service event-store-service notification-service \
  api-gateway webapp
```

**Démarrer uniquement le monitoring :**

```
just docker-monitoring
# ou
docker compose up -d prometheus grafana
```

#### 9.2.5 Vérification du déploiement

1. **Vérifier l'état des conteneurs :**

   ```
   docker compose ps
   ```

   Tous les services doivent être en état `Up (healthy)`.

2. **Vérifier les logs :**

   ```
   just docker-logs
   # ou
   docker compose logs -f
   ```

3. **Tester les endpoints de santé :**

   ```
   curl http://localhost:8080/v1/health
   curl http://localhost:3001/v1/health
   curl http://localhost:3002/v1/health
   curl http://localhost:3003/v1/health
   curl http://localhost:3004/v1/health
   ```

   Chaque endpoint doit retourner `{"status": "ok"}`.

4. **Accéder à l'interface web :**
   - Web App : `http://localhost:8000`

### 9.3 Surveillance et monitoring

#### 9.3.1 Prometheus

**URL :** `http://localhost:9090`

#### 9.3.2 Grafana

**URL :** `http://localhost:3000`

**Identifiants par défaut :**

- Username : `admin`
- Password : `admin123`

**Configuration initiale :**

1. Se connecter à Grafana
2. Importer le dashboard pré-configuré :

- Aller dans Dashboards → Import
- Télécharger `./grafana_dashboard.json`
- Sélectionner la source Prometheus
- Cliquer sur Import

**Dashboards disponibles :**
- Performance globale (latence, débit)
- Statut de la queue d'ordres
- Métriques par service
- Erreur 4xx et 5xx
- 4 Golden Signals

#### 9.3.3 Logs

**Consulter les logs en temps réel :**

```
# Tous les services
docker compose logs -f

# Service spécifique
docker compose logs -f api-gateway
docker compose logs -f event-store-service
docker compose logs -f notification-service

# Filtrer par niveau
docker compose logs -f | grep ERROR
docker compose logs -f | grep WARN
```

**Structure des logs :**

Les logs utilisent le format structuré `tracing` de Rust avec les niveaux suivants :
- `ERROR` : Erreurs critiques nécessitant une action immédiate
- `WARN` : Situations anormales mais gérées
- `INFO` : Événements importants du système
- `DEBUG` : Informations de débogage détaillées (désactivé en production)
- `TRACE` : Traçage très détaillé (désactivé en production)

### 9.4 Tests de charge

#### 9.4.1 Préparation

1. **Activer le bypass MFA :**

   Éditer `docker-compose.yml` et définir `MFA_BYPASS=true` dans la section API Gateway :

   ```
   api-gateway:
     environment:
       - MFA_BYPASS=true
   ```

2. **Redémarrer les services :**

   ```
   docker compose up -d api-gateway
   ```

#### 9.4.2 Exécution des tests K6

**Configuration :**

```
just load-test-setup
```

**Lancer les tests :**

```
just load-test
# ou
cd load-tests && k6 run scenarios/quality-validation.js
```

**Scénarios disponibles :**

- `quality-validation.js` : Test de charge complet (800+ ordres/s)
- `market-data-consultation.js` : Test de consultation des données de marché

**Métriques attendues :**

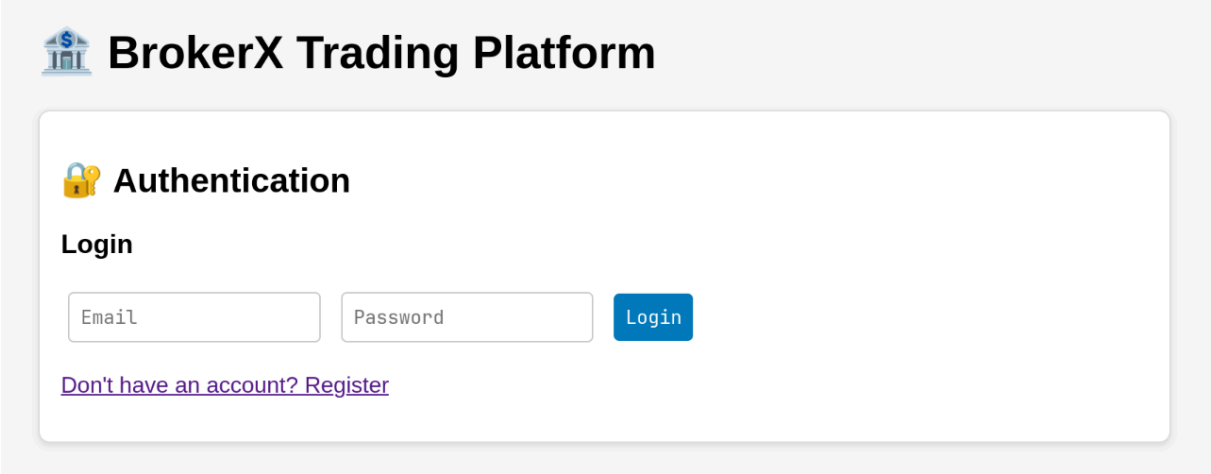
- Débit : ≥ 800 ordres/s
- Latence P95 : ≤ 250 ms
- Taux d'erreur : < 1%

# 10 Guide d'utilisation

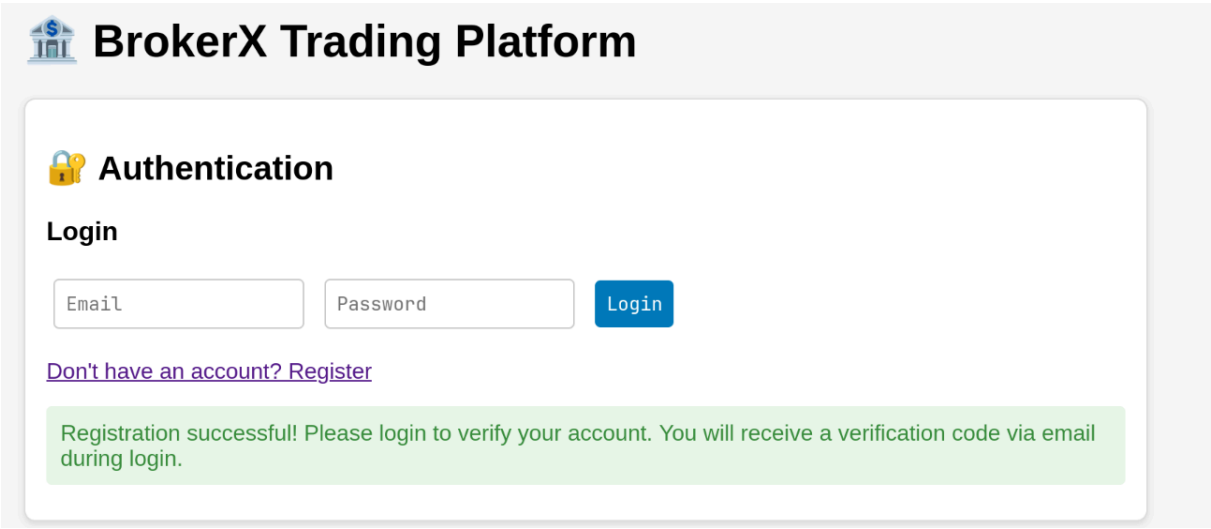
## 10.1 Frontend Web

Créer un nouveau compte utilisateur:

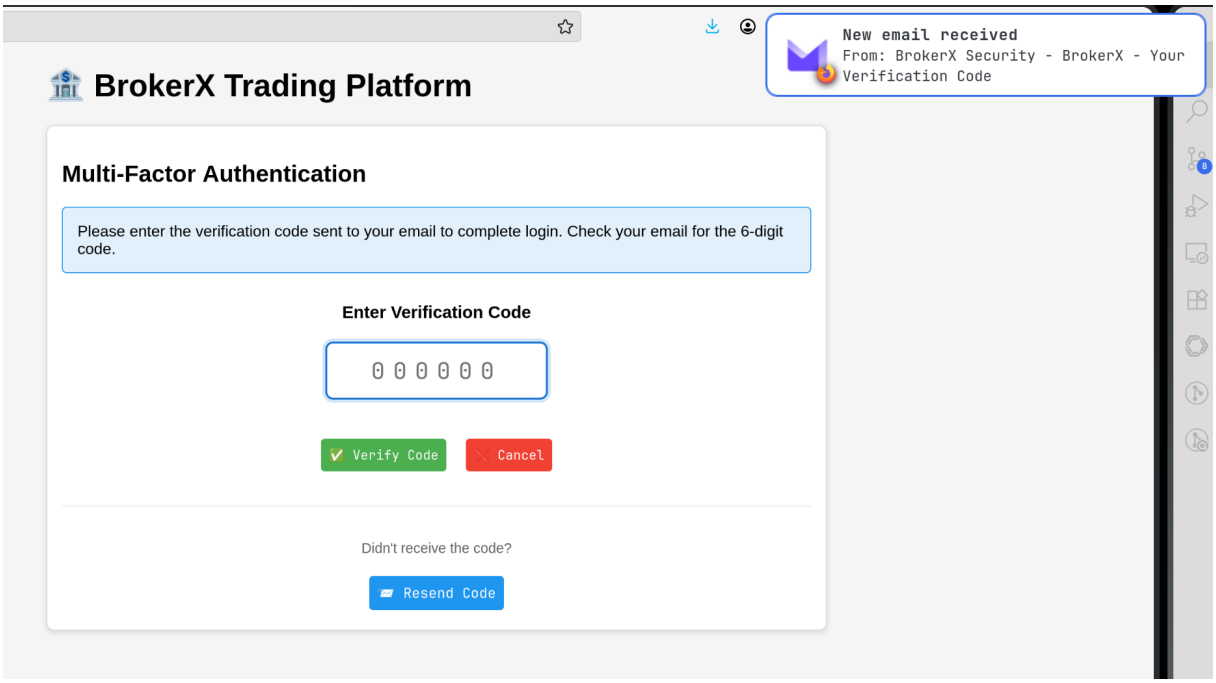
On peut cliquer sur « Register »:



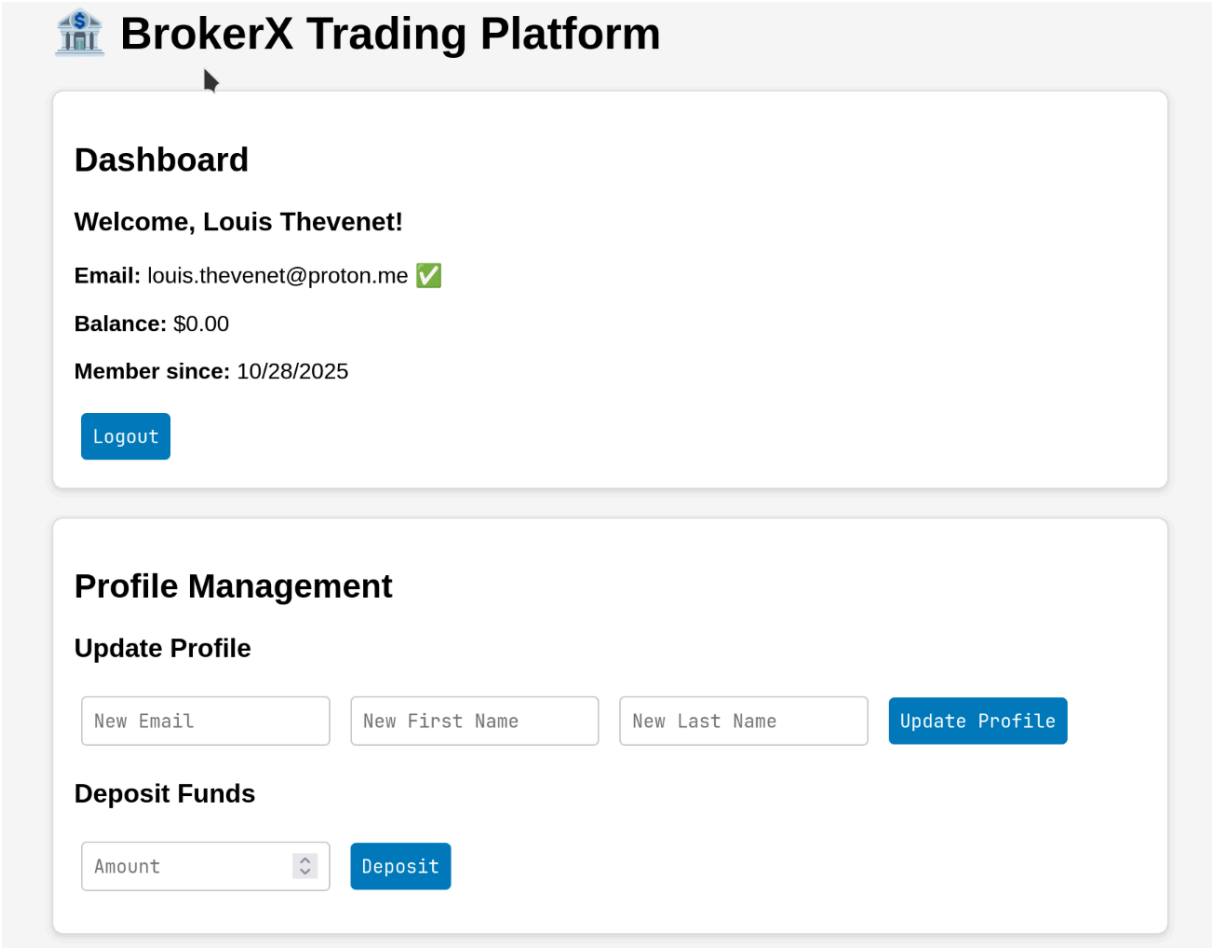

Après avoir rempli le formulaire d'inscription:



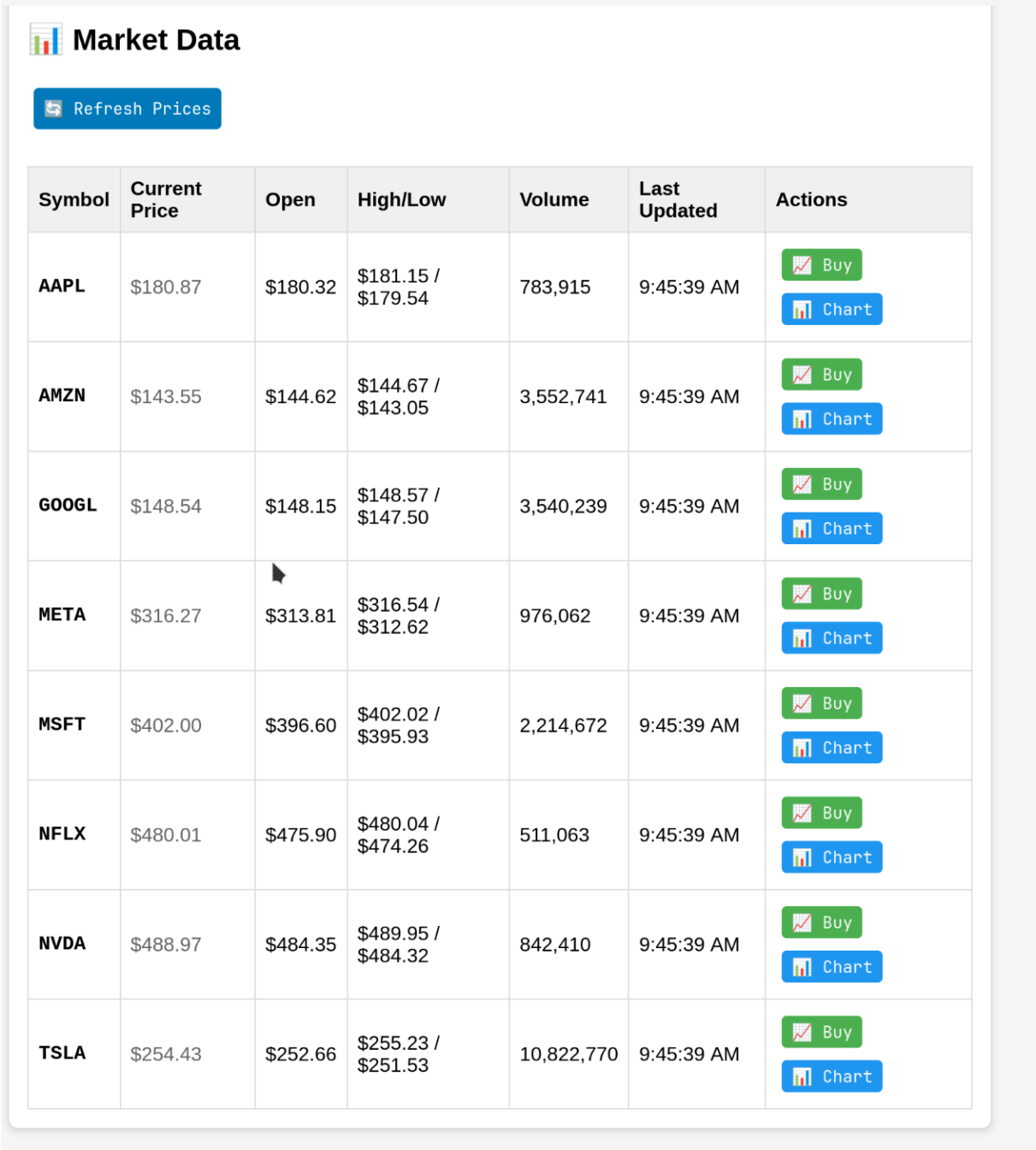

On peut maintenant se connecter:



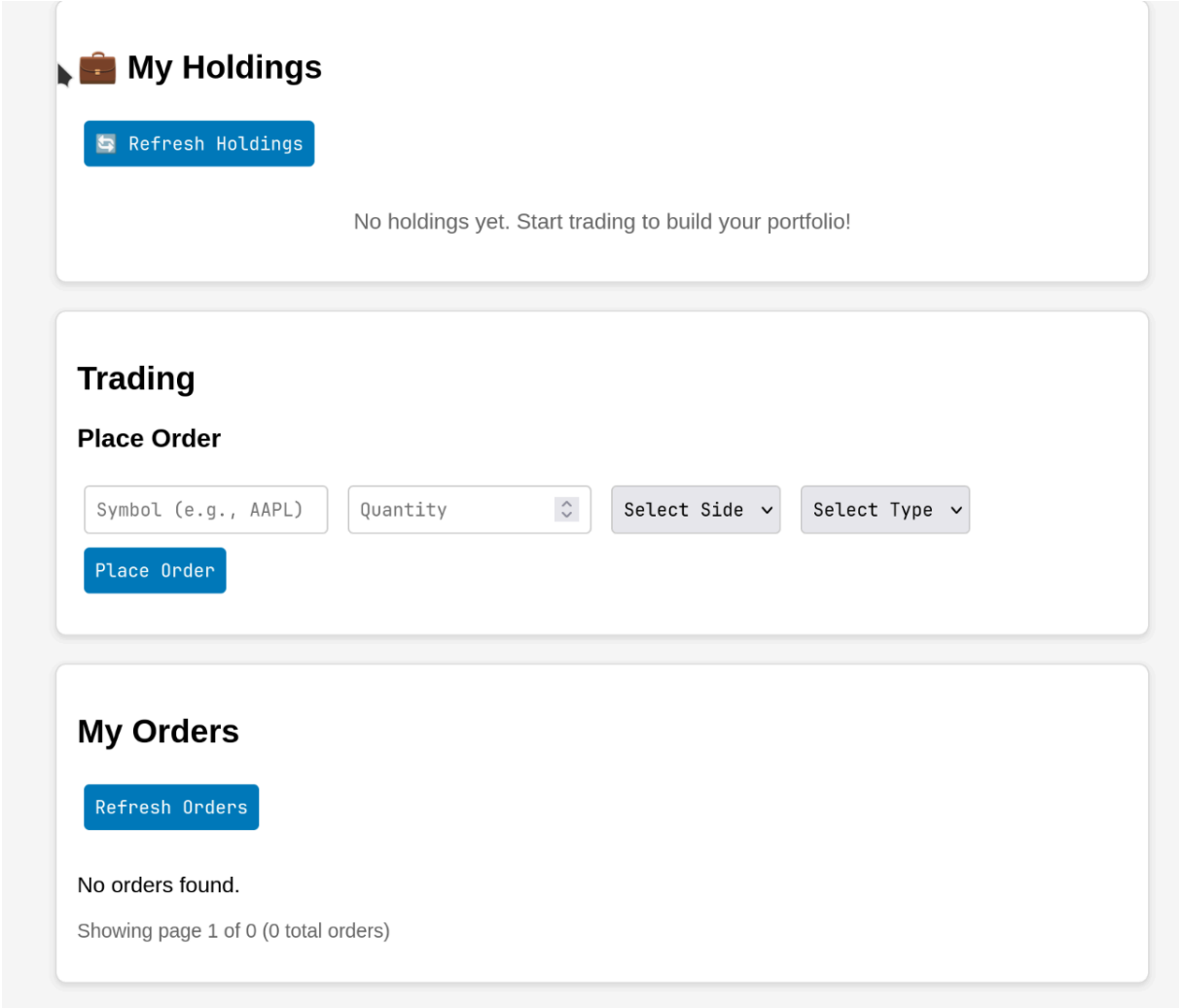

Vue informations personnelles:



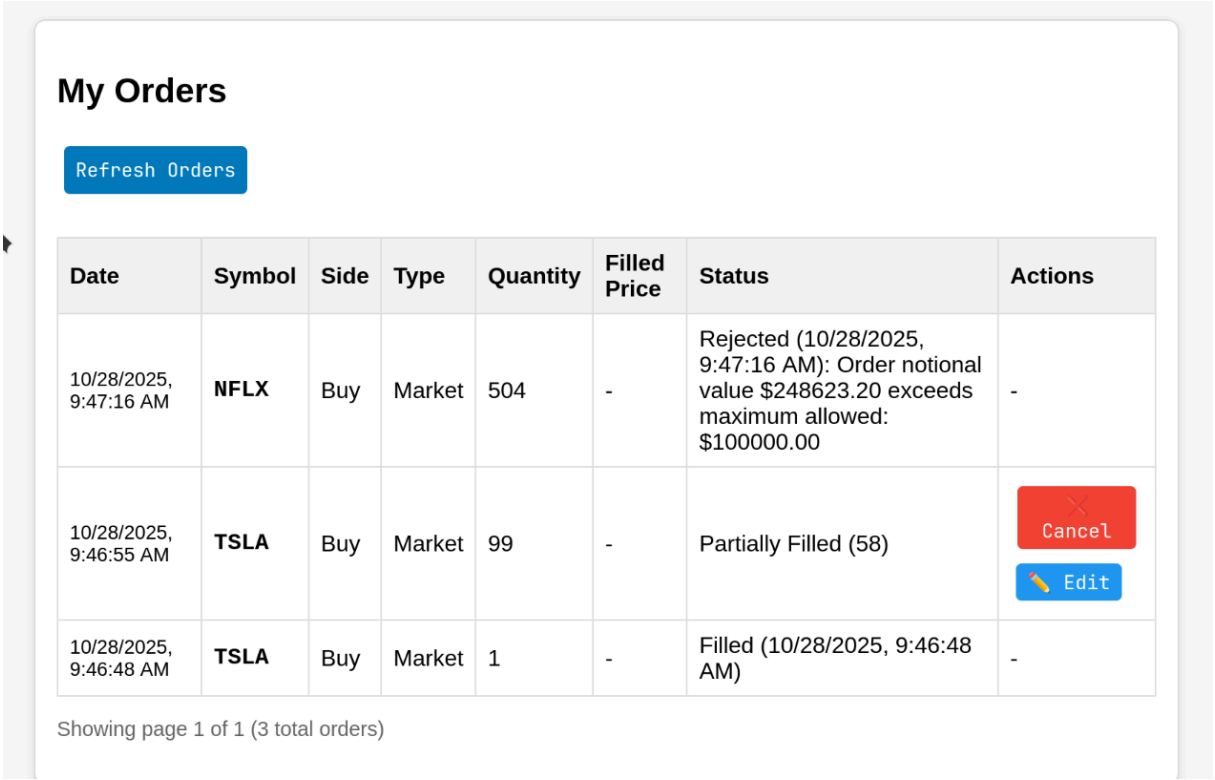

Vue marché:

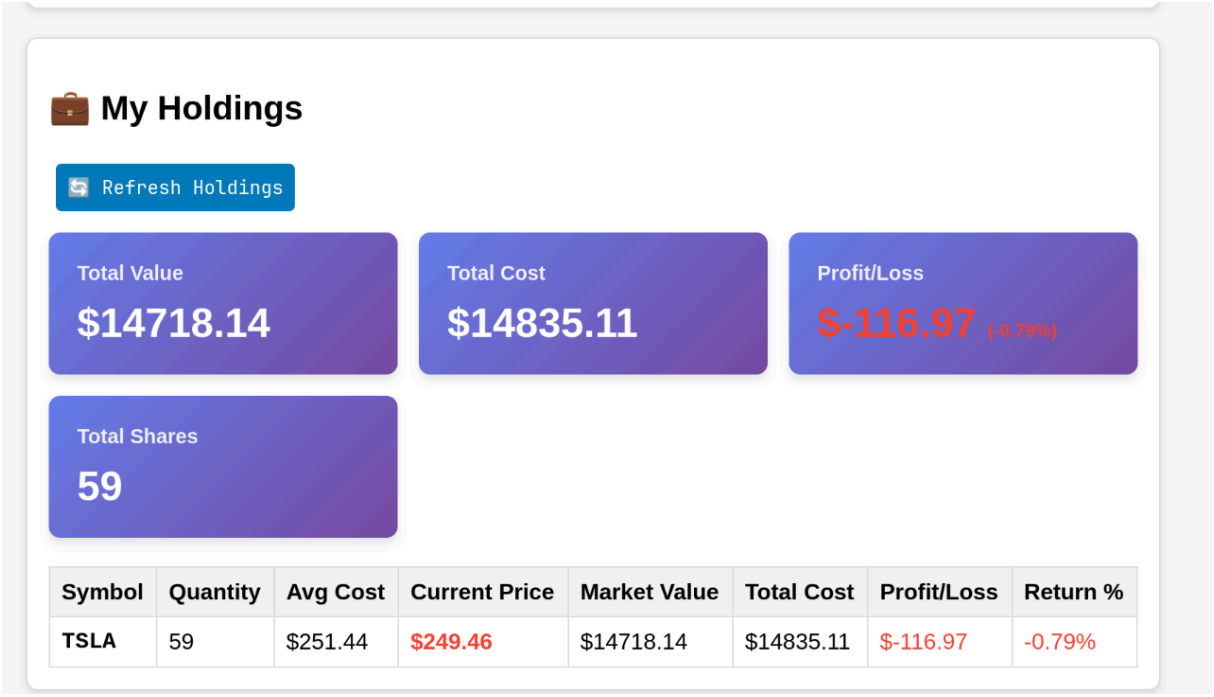



Portfolio, ordres passés et formulaire de passage d'ordre:




Après avoir passé des ordres:




## 10.2 API

### 10.2.1 Swagger UI

**API Gateway** http://127.0.0.1:8080/swagger-ui/
**User Service** http://127.0.0.1:3001/swagger-ui/
**Order Service** http://127.0.0.1:3002/swagger-ui/
**Market Data Service** http://127.0.0.1:3003/swagger-ui/
**Event Store Service** http://127.0.0.1:3005/swagger-ui/

**Notification Service** http://127.0.0.1:3006/swagger-ui/

### 10.2.2 Postman

Les collections sont également disponibles dans le répertoire `docs/postman/`.
**API Gateway** http://127.0.0.1:8080/api-docs/openapi.json
**User Service** http://127.0.0.1:3001/api-docs/openapi.json
**Order Service** http://127.0.0.1:3002/api-docs/openapi.json
**Market Data Service** http://127.0.0.1:3003/api-docs/openapi.json
**Event Store Service** http://127.0.0.1:3005/api-docs/openapi.json
**Notification Service** http://127.0.0.1:3006/api-docs/openapi.json

# 11 Glossaire

| Terme | Définition |
|---|---|
| Ordre | Instruction d'achat ou de vente d'un instrument financier |
| Portefeuille | Ensemble des actifs financiers détenus par un utilisateur |
| MFA | Authentification multi-facteurs |
| OTP | One-Time Password, mot de passe à usage unique |
| JWT | JSON Web Token, format de token sécurisé pour l'authentification |